LOVELY PROFESSIONAL UNIVERSITY

College Project Report

StoneDesk: Dispatch and Billing Management System

A web-based multi-tenant system for stone-load operations, billing, and business administration

Submitted by

Siva Pavan Kumar Reddy Lingala

Register Number: K23WS - 34

Academic Year

2026–2027

Declaration

I hereby declare that this project report titled **“StoneDesk: Dispatch and Billing Management System”** is a true summary of the work carried out for academic purposes. The implementation, design decisions, testing approach, and technical discussion presented in this report are based on the project source code and supporting project documentation.

Student Signature	Date

Acknowledgement

I would like to express my sincere gratitude to Lovely Professional University for providing the academic environment and resources needed to complete this project. I also thank the faculty members and project reviewers for their guidance and feedback.

I am grateful to everyone who contributed suggestions during the design, implementation, database integration, and testing stages of StoneDesk.

Abstract

StoneDesk is a web-based dispatch and billing management system designed for businesses that handle stone loads, trucks, destinations, loading requirements, and customer invoices. The system replaces fragmented manual workflows with a centralized application that supports authenticated users, organization-level data isolation, draft and finalized dispatches, loading lists, invoice payments, saved business references, audit logging, and business settings.

The backend is implemented with Node.js and Express. MongoDB is used as the persistence layer through Mongoose schemas and a compatibility data adapter. The frontend is implemented as a React single-page application. The system follows a multi-tenant model in which each organization owns its settings, users, dispatch records, loading lists, payments, and audit history. Server-side validation and calculation ensure that billing totals are not trusted from the browser.

Keywords: MongoDB, Mongoose, Express, React, multi-tenancy, dispatch management, billing, REST API, authentication.

Table of Contents

Introduction 1

Existing System and Literature Review 2

Requirements and Feasibility 3

System Architecture 4

MongoDB and Database Design 5

Implementation Details 7

Testing and Validation 9

Security and Multi-Tenancy 11

Results and Discussion 12

Conclusion and Future Scope 13

References 14

1. Introduction

1.1 Background

Stone and construction-material businesses frequently manage truck movements, customer destinations, measurements, material rates, invoices, and payments at the same time. When these activities are handled through paper records or disconnected spreadsheets, errors can occur in totals, status tracking, historical branding, and access control.

StoneDesk addresses this problem through a single web application. A user can create a draft load, enter inventory measurements, calculate totals, finalize a dispatch, update delivery status, record payments, and maintain organization settings.

1.2 Objectives

- Digitize stone dispatch and billing operations.
- Provide role-based access for administrators, yard managers, and dispatchers.
- Maintain strict organization-level data isolation.
- Calculate measurements and totals on the server.
- Support draft editing, finalization, revision history, and payment tracking.
- Provide searchable and paginated lists for operational records.

1.3 Scope

The scope includes authentication, business profile settings, product rates, dispatches, loading lists, invoice payments, audit records, logo storage, and responsive frontend workflows. It does not attempt to replace accounting software or external payment gateways.

2. Existing System and Literature Review

2.1 Existing approach

In a manual workflow, dispatch details may be recorded in notebooks, spreadsheets, or separate messaging channels. Such workflows make it difficult to guarantee that totals are consistent, that only authorized staff can edit records, and that historical invoices retain the correct business information.

2.2 Proposed approach

StoneDesk introduces a centralized REST API and a browser-based interface. MongoDB stores flexible operational documents, while Mongoose schemas provide validation, indexes, and persistence. The application adds server-side calculations, optimistic concurrency checks, organization filters, and audit events.

2.3 Technology selection

Technology	Role in the project	Reason for use
React	Frontend user interface	Component-based interface and responsive single-page workflows.
Node.js and Express	Backend API	Lightweight HTTP server and a clear controller/router structure.
MongoDB	Database	Document storage suited to nested inventory and revision data.
Mongoose	ODM	Schema definitions, validation, indexes, and MongoDB connectivity.
Node test runner	Testing	Built-in test execution without adding a separate test framework.

3. Requirements and Feasibility

3.1 Functional requirements

- Users can register, log in, reset passwords, and maintain sessions.
- Administrators can manage business settings, products, staff, and roles.
- Authorized users can create, edit, finalize, revise, and delete dispatches according to status and role.
- Users can create loading lists and track Draft, Loading, and Completed states.
- Authorized users can record invoice payments.
- The system can search, filter, and paginate operational records.

3.2 Non-functional requirements

- Data isolation between organizations.
- Validation of user input and server-side financial calculations.
- Reliable error responses in JSON format.
- Responsive usability across desktop and mobile widths.
- Indexes for common organization and date/status queries.

3.3 Feasibility

Technical feasibility: The project uses widely supported JavaScript technologies and a document database that handles nested operational data naturally.

Operational feasibility: The workflow reflects common activities such as preparing a load, finalizing an invoice, and recording delivery.

Economic feasibility: The application can run using standard Node.js hosting and MongoDB hosting options, reducing the need for specialized infrastructure.

4. System Architecture

The frontend communicates with Express API routes. Middleware performs origin checks, JSON parsing, sessions, and authorization. Controllers apply business rules and call the Mongoose data adapter. The adapter translates Prisma-style operations into Mongoose queries and persists documents in MongoDB.

5. MongoDB and Database Design

The database layer is implemented using MongoDB and Mongoose. Connection lifecycle management is located in `Backend/utils/db.js`. The application reads `DATABASE_URL` and otherwise uses a local MongoDB URI for the `stonedesk` database.

5.1 Main collections

Collection/model	Purpose	Important fields
MasterSettings	Organization root	organizationId, businessName, savedTrucks, savedDestinations, stoneRates, version
User	Organization members	organizationId, email, passwordHash, role, active
Dispatch	Truck load and invoice record	dispatchSlipNumber, logistics, inventory, summary, status, revisions
LoadingList	Loading requirements	loadingListNumber, requirements, status
InvoicePayment	Payments on finalized bills	dispatchId, amount, note, createdByUserId
Session	Cookie sessions	tokenHash, userId, expiresAt
AuditLog	Security and business history	organizationId, userId, action, timestamp, details

5.2 Indexing strategy

Compound indexes begin with `organizationId` because most operational queries are tenant-scoped. Dispatch and loading-list numbers are unique within an organization. Date and status indexes support recent-record lists and dashboard queries. Session expiration uses a MongoDB TTL index on `expiresAt`.

```
DispatchSchema.index({ organizationId: 1, dispatchSlipNumber: 1 }, { unique: true });
DispatchSchema.index({ organizationId: 1, status: 1, createdAt: -1 });

SessionSchema.index({ expiresAt: 1 }, { expires: 0 });
```

5. MongoDB and Database Design (continued)

5.3 Entity relationship view

5.4 Data access adapter

`Backend/utils/prisma.js` exposes methods such as `findFirst`, `findMany`, `create`, `updateMany`, `deleteMany`, and `count`. It translates conditions such as `in`, `contains`, and `OR` into MongoDB operators including `$in`, `$regex`, and `$or`. This preserves controller readability while using Mongoose under the hood.

6. Implementation Details

6.1 Dispatch lifecycle

1. **Draft creation:** The server validates metadata and inventory, calculates totals, and stores a Draft document.
2. **Draft update:** The record is updated only if it is still Draft and its `updatedAt` matches the client version.
3. **Finalization:** The server recalculates inventory and totals, creates a business snapshot, and changes status to Dispatched.
4. **Delivery:** Authorized roles can move a record from Dispatched to Delivered.
5. **Revision:** Finalized records preserve the previous values in a revisions array before applying new data.

6.2 Server-side calculations

The server validates dimensions, quantities, rates, and measurement rows. Area is calculated from length, width, and quantity. Totals are generated on the server so a modified client cannot submit a false invoice amount.

6.3 Business settings

Master settings store business profile information, products, royalty defaults, trucks, and destinations. Saved references are deduplicated, ordered newest first, capped, and updated without causing a dispatch save to fail.

6.4 API and error handling

Express routes delegate work to controllers. Controllers return JSON errors with appropriate HTTP status codes. Unexpected errors are forwarded to the central error handler. List endpoints use projections to avoid returning unnecessary fields.

7. Testing and Validation

7.1 Testing tools

The project uses Node.js built-in `node:test` and `node:assert/strict`. Backend tests run sequentially through:

```
npm --prefix Backend test
node --test --test-concurrency=1 tests/*.test.js
```

The root command runs both backend and frontend suites:

```
npm test
```

7.2 Backend integration testing

Backend tests create an HTTP server from the Express application and call routes using `fetch()`. The tests connect to MongoDB through the same Mongoose connection used by the application and access records through the adapter.

7.3 Test coverage areas

Area	**Validation performed**
Authentication	Signup, login, cookies, sessions, recovery, paused users.
Dispatches	Creation, draft updates, finalization, delivery, revision, deletion, idempotency.
Multi-tenancy	Organization A cannot read or modify organization B records.
Authorization	Admin, YardManager, and Dispatcher role behavior.
Database behavior	Projections, persistence, payments, settings, invitations, audit logs.
Calculations	Measurement validation, totals, rates, and frontend/backend parity.
Frontend	Invoices, exports, drafts, routes, forms, security policy, responsive UI.

8. Security and Multi-Tenancy

8.1 Tenant isolation

Every organization-owned query includes the current user's `organizationId`. A record belonging to another organization is treated as unavailable and returns a 404 response instead of exposing information about its existence.

8.2 Authentication and sessions

Passwords are stored as derived hashes with salts. Browser sessions use an HTTP-only cookie, while MongoDB stores a hash of the session token. Sessions expire through a TTL index.

8.3 Authorization

Middleware checks whether the user has a valid session, is active, and has the required role. Status transitions and finalized invoice operations apply additional role rules in controllers and shared utilities.

8.4 Input and data safety

- Request bodies are validated before persistence.
- Totals are calculated on the server.
- Regular expressions are escaped before being used for search.
- Queries use projections for list responses.
- Audit logs record important security and business events.

> **Security principle:** authentication identifies the user, authorization checks the role, and tenant scoping checks the organization. All three are needed for a safe multi-tenant API.

9. Results and Discussion

9.1 Achieved results

- A complete browser-based workflow for dispatch and billing operations.
- Centralized MongoDB persistence through Mongoose models.
- Organization-level isolation across users, dispatches, loading lists, and payments.
- Server-authoritative measurements and invoice calculations.
- Operational support for draft, finalized, delivered, and revised records.
- Structured automated test coverage across backend and frontend behavior.

9.2 Discussion

The document model is particularly suitable for inventory and revision data because the nested structures can be stored together with the dispatch. Compound organization indexes support the application’s access pattern. The compatibility adapter reduces migration risk, but it also means that developers must understand both the adapter contract and the underlying Mongoose behavior.

The project demonstrates that database correctness is not only about storing documents. It also depends on tenant filters, concurrency checks, validation, indexes, authorization, and tests that exercise the complete HTTP path.

10. Conclusion and Future Scope

10.1 Conclusion

StoneDesk provides a practical foundation for digitizing stone dispatch and billing operations. Its architecture separates the frontend, API, controllers, data adapter, and MongoDB models. The use of Mongoose schemas, organization-based indexes, server-side calculations, role checks, and integration tests improves reliability and maintainability.

10.2 Future scope

- Introduce a dedicated reporting dashboard with exportable organization analytics.
- Add stronger production observability for slow MongoDB queries and API latency.
- Propagate Mongoose sessions explicitly through every operation that participates in a transaction.
- Add configurable tax, invoice numbering, and accounting-system integrations.
- Support cloud object storage for production logo and document assets.
- Provide mobile-focused offline support for yard operations.
- Expand automated accessibility and browser compatibility testing.

References

1. StoneDesk project source code, `Backend/` and `Frontend/` directories.
2. StoneDesk Technical Architecture, `docs/ARCHITECTURE.md` .
3. StoneDesk Business Workflows, `docs/BUSINESS-WORKFLOWS.md` .
4. StoneDesk backend conventions, `.github/instructions/backend.instructions.md` .
5. MongoDB Documentation, “MongoDB Manual,” MongoDB Inc.
6. Mongoose Documentation, “Mongoose ODM for MongoDB,” Automattic.
7. Node.js Documentation, “Test runner,” Node.js Foundation.
8. Express.js Documentation, “Express web framework,” OpenJS Foundation.
9. React Documentation, “React,” Meta Open Source.

This report was generated from the StoneDesk project structure and source documentation. It is intended as a college-level technical report and can be supplemented with institution-specific certification, signatures, and supervisor approval pages.